

Understanding Experimental Data

This lecture opens the bridge between computation and the physical and social sciences: how we take imperfect measurements from the real world and use computational tools — starting from Hooke’s law and the humble spring — to fit models to them.

Statistics Meets Experimental Science: The Basic Recipe

The professor frames the whole topic as a recurring three-step pattern connecting statistics to experimental science:

1. **Conduct an experiment to gather data.** The experiment may be *physical* (something done in a biology lab) or *social* (handing out questionnaires to people).
2. **Use theory to generate questions about that data.** Again, the theory can be physical (e.g., about gravitational fields) or social (e.g., the fact that people give inconsistent answers when asked questions).
3. **Design a computation to help answer those questions about the data.**

A handwritten example on the slides shows the pattern concretely: *what is the net gain on a missed jump shot in basketball?* It is written as

$$\text{net gain} = P(\text{offensive rebound}) \cdot E[\text{points for}] - P(\text{defensive rebound}) \cdot E[\text{points against}]$$

Theory supplies the question and its structure; computation evaluates it against data. Everything in this unit builds toward step 3 — designing computations that answer theoretically motivated questions about noisy measurements.

Hooke’s Law: The Physics of a Linear Spring

Two very different springs

To make things concrete, the lecture contrasts two springs: the suspension spring on a motorcycle, with a spring constant of roughly 35,000 N/m, and a Slinky, with a spring constant of only about 1 N/m — an enormous difference in stiffness. The key notion is a **linear spring**: the amount of force needed to stretch or compress the spring is *linear* in the distance it is stretched or compressed, and each spring has a spring constant k determining how much force is needed.

Units matter here: a **newton** is the force needed to accelerate a 1 kilogram mass by 1 meter per second per second.

The law itself

Per the underlying physics, Hooke’s law is an **empirical law**: the force F needed to extend or compress a spring by some distance x scales linearly with that distance,

$$F_s = kx,$$

where k is a positive constant characteristic of the spring (its *stiffness*), valid for displacements x small compared to the total possible deformation. It is named after the 17th-century British physicist Robert Hooke, who first stated it in 1676 as a Latin anagram and published the solution in 1678 as *ut tensio, sic vis* — “as the extension, so the force” — noting he had been aware of it since 1660.

Two equivalent sign conventions exist, and the distinction matters:

- If F_s is the **applied** force pulling the free end of a helical spring (one end fixed, the other displaced by x from its relaxed position at equilibrium), then $F_s = kx$, or equivalently $x = F_s/k$. For compression, the same formula holds with both F_s and x negative.
- If F_s is the **restoring force** exerted *by* the spring on whatever pulls its free end, the equation becomes

$$F_s = -kx,$$

because the restoring force acts in the direction *opposite* to the displacement. This is the form used in lecture: stretch the spring and it pulls back; compress it and it pushes out.

Geometrically, the graph of applied force versus displacement is a **straight line passing through the origin whose slope is k** — which is exactly what makes the problem of recovering k from data a line-fitting problem later in the unit.

Where the law applies

An elastic body or material for which this equation can be assumed is said to be **linear-elastic** or **Hookean**. The law is the fundamental principle behind the spring scale, the manometer, the galvanometer, and the balance wheel of the mechanical clock. It extends far beyond simple coil springs, applying to any elastic object of arbitrary complexity as long as deformation and stress can each be expressed by a single signed number:

- A block of rubber attached to two parallel plates and deformed by **shearing** obeys Hooke’s law (for small enough deformations) relating shearing force to sideways plate displacement.
- A straight steel bar or concrete beam supported at both ends and **bent** by a weight at an intermediate point obeys it too, with x the transverse deviation from the unloaded shape.
- The **torsional analog** states that the torque τ required to rotate an object is directly proportional to the angular displacement θ from equilibrium, again with a negative sign indicating a restoring torque.

In full generality, the modern theory of elasticity says the **strain** of an elastic object is proportional to the **stress** applied to it. Because general stresses and strains have multiple independent components, the “proportionality factor” becomes a linear map (a tensor, representable as a matrix of real numbers) rather than a single number. In this form one can deduce, for example, that a homogeneous rod with uniform cross section behaves like a simple spring whose stiffness k is directly proportional to its cross-sectional area and inversely proportional to its length.

Worked Example: How Heavy Must a Rider Be?

Using the motorcycle spring ($k \approx 35,000$ N/m), the question is: how much does a rider have to weigh to compress the spring by 1 centimeter?

Convert to meters: $d = 0.01$ m. The required force magnitude is

$$F = kd = 0.01 \text{ m} \times 35,000 \text{ N/m} = 350 \text{ N}.$$

Force is also mass times acceleration, and the relevant acceleration here is gravity, so $F = mg$ with $g = 9.8 \text{ m/s}^2$. Solving,

$$m = \frac{350 \text{ N}}{9.81 \text{ m/s}^2} \approx 35.68 \text{ kg}.$$

Notation warning flagged in lecture: in “35.68k,” the little “k” stands for *kilograms*, not the spring constant — don’t let the symbol collision confuse you.

Limits of Hooke’s Law: When the Model Breaks

It is crucial to understand that Hooke’s law is a **first-order linear approximation** to the real response of springs and elastic bodies. It fails once forces exceed some limit: no material can be compressed beyond a certain minimum size, or stretched beyond a maximum size, without permanent deformation or change

of state. Moreover, many materials deviate noticeably from Hooke's law *well before* those elastic limits are reached.

This is visible in the stress–strain curve for low-carbon steel: Hooke's law is valid only for the portion of the curve between the origin and the yield point. Beyond yield, the material enters strain hardening, necking, and eventually rupture — behavior no linear model can capture.

Why does this matter for the theme of the lecture? Because even with perfect measurements, a linear model is only correct within a limited operating range — model validity and measurement quality are separate concerns, and both must hold for data fitting to succeed.

Experimental Error: Why Real Measurements Differ from Theory

Here is the pivot of the lecture: what happens when we actually go into the lab, hang weights off a spring, and measure displacements ourselves? The measurements will not be perfect.

Observational error (or measurement error) is defined as the difference between a measured value of a quantity and its unknown true value. Such errors are inherent in the measurement process itself — lengths measured with a ruler calibrated in whole centimeters will carry a measurement error of several millimeters simply because of the ruler's granularity. The error or uncertainty of a measurement can be estimated and is specified alongside the value, e.g., 32.3 ± 0.5 cm.

Scientific observations are marred by **two distinct types of error**, and telling them apart is essential because they behave completely differently under repetition:

Random error

Random (statistical) error is always present in a measurement. It is caused by inherently unpredictable fluctuations in the readings of the apparatus or in the experimenter's interpretation of the reading, possibly partly due to interference of the environment with the measurement process. Its signature properties:

- Repeated measurements of a constant quantity give **inconsistent values**, and these errors are **uncorrelated between measurements**.
- They can be **estimated by comparing multiple measurements**; in a large set of measurements, a **standard deviation** can be calculated as an estimate of the amount of statistical error.
- They can be **reduced by averaging** multiple measurements — this is the key practical mitigation.
- Random error is closely tied to **precision**: the higher the precision of an instrument, the smaller the variability (standard deviation) of its fluctuating readings.

Systematic error

Systematic error is not determined by chance; it is introduced by **repeatable processes inherent to the system**. It is predictable and typically constant or proportional to the true value, and it always affects results in a predictable direction. Sources include:

- Imperfect calibration of measurement instruments — incorrect zeroing (“zero error”) is the classic instrumentation example;
- Changes in the environment interfering with the measurement process;
- Imperfect methods of observation;
- Uncertainty in correction terms applied during experimental analysis, and errors from using approximate theoretical models.

Its signature property is the mirror image of random error's: **repeated identical measurements do not reduce systematic error**, because every measurement is altered in the same direction. Instead, it must be carefully avoided or identified — if the cause can be identified, it can usually be eliminated, and it may often be reduced with standardized procedures. Part of learning any science is mastering standard instruments and protocols to minimize it; over long periods, systematic errors get resolved into a form of “negative knowledge”

— accumulated understanding of how to avoid specific kinds of error. Systematic error is sometimes called **statistical bias**.

A fully worked illustration from the source: an experimenter timing a pendulum swinging past a fiducial marker whose stopwatch starts with 1 second already on the clock will have *all* results off by 1 second (zero error). Repeating the experiment twenty times does not fix this — the calculated average retains a percentage error, coming out slightly larger than the true period.

Accuracy versus precision

These two error types map onto two distinct quality dimensions of a measurement:

- A ruler accurately calibrated in whole centimeters suffers **random error** with each use — the same distance yields slightly different values — resulting in limited **precision**.
- A metallic ruler whose temperature is not controlled suffers thermal expansion, adding a **systematic error** on top, resulting in limited **accuracy**.

So precision is about the *spread* of repeated measurements (random error), while accuracy is about closeness to the true value (which systematic error destroys).

The combined statistical picture

The common statistical model treats the error as having **two additive parts** — a random component and a systematic component. Note that some errors fall into neither category cleanly, such as the uncertainty in the calibration of an instrument. And when two or more observations or instruments are combined, their errors combine: the error estimate for the result depends on the statistical characteristics of each individual measurement *and* on any statistical correlation between them.

The following figure makes the decomposition visual: the cloud of measurements scatters randomly around a center that is itself offset from the true value by the constant systematic error.

From Physics to Data Fitting

Pulling the threads together: we have a clean theoretical model — Hooke’s law, $F = -kd$, a straight line with slope k — and a lab procedure (hang weights, measure displacement) that produces data corrupted by both random and systematic error. The remainder of this unit is about the computational side of the recipe: how to **fit models like Hooke’s law to noisy experimental data**, so that theory-generated questions can be answered despite imperfect measurements.

Measuring a Spring Constant with Hooke’s Law

The physical motivation for this part of the lecture is **finding** k , the spring constant of a spring. Hooke’s law is an *empirical* law of physics: the force needed to extend or compress a spring by some distance scales linearly with that distance,

$$F_s = kx,$$

where k is a positive constant characterizing the spring’s stiffness and x is small compared to the spring’s total possible deformation. The law is named after the 17th-century British physicist Robert Hooke, who first stated it in 1676 as a Latin anagram and published the solution in 1678: *ut tensio, sic vis* — “as the extension, so the force” — though he claimed awareness of it since 1660. It is the fundamental principle behind the spring scale, the manometer, the galvanometer, and the balance wheel of the mechanical clock.

In the form used in lecture, $F = -k \cdot d$: the minus sign reflects the convention that F is the *restoring* force exerted by the spring, directed opposite to the displacement d from rest position. Rearranging gives $k = -F/d$. The lab procedure exploits equilibrium: hang a mass m from a vertical spring; gravity pulls

down with force $F = mg$, the spring stretches by d , and at equilibrium the restoring force exactly balances the weight. Working in magnitudes,

$$k = \frac{mg}{d} = \frac{9.81 \cdot m}{d},$$

so knowing the hung mass and measuring the stretch suffices to compute k . A useful geometric fact from the theory: graphing applied force against displacement yields a straight line through the origin whose slope is k — which is precisely why fitting a line to force–displacement data is the right way to estimate k .

Two caveats from the physics matter later. First, Hooke’s law is a *first-order linear approximation*: it fails once forces exceed some limit, since no material can be stretched beyond a maximum size without permanent deformation, and many materials deviate noticeably well before that elastic limit. Second, the modern theory of elasticity generalizes the law — strain proportional to stress, with the “proportionality factor” becoming a tensor (matrix) when stresses and strains have multiple independent components. Even in that general form, familiar objects reduce to simple springs: a homogeneous rod of uniform cross section has a stiffness k directly proportional to its cross-sectional area and inversely proportional to its length. Analogues abound — torsional springs ($\tau \propto \theta$, again with a restoring minus sign), rubber blocks deformed in shear between parallel plates, and beams bent by a weight — all obeying the same scalar proportionality as long as deformation and stress can each be captured by a single signed number.

The Experimental Data: Thirteen Messy Measurements

Running the experiment produces thirteen (mass, displacement) pairs:

Mass (kg)	Displacement (m)
0.10	0.0865
0.15	0.1015
0.20	0.1106
0.25	0.1279
0.30	0.1892
0.35	0.2695
0.40	0.2888
0.45	0.2425
0.50	0.3465
0.55	0.3225
0.60	0.3764
0.65	0.4263
0.70	0.4562

This is real experimental data, so it is messy: displacement does not increase monotonically with mass. The lecture highlights that 0.45 kg produced a *smaller* stretch than 0.40 kg (0.2425 m vs. 0.2888 m); the pair 0.50/0.55 kg shows the same kind of dip. Such reversals are measurement noise, and they are exactly why a naive “connect the dots” reading of the data will not do — we need principled fitting.

From Masses to Forces: Processing and Plotting the Data

The `plotData` function turns the raw file into a force–displacement plot. It takes a file name, calls `getData` to read in the x and y values, converts both to pylab arrays so arithmetic can be performed on them elementwise, and then applies the key transformation: multiplying every x value by 9.81. Since the stored x values are masses, multiplication by $g = 9.81 \text{ m/s}^2$ converts them to forces in Newtons via $F = mg$. The points are then plotted as blue circles (`'bo'`) labeled “Measured displacements,” with `labelPlot` handling the axis labels and title.

The resulting picture spans forces from 0 to 10 N horizontally and distances from 0.05 to 0.50 m vertically. What it shows matches theory only approximately: distance increases roughly linearly with force, as Hooke’s law predicts for a line through the origin, but the points wobble around any straight path, and at the high-force end — around 8–9 N — the curve visibly flattens out near 0.45 m, meaning the spring stops stretching proportionally. This is exactly the behavior the physics anticipates: Hooke’s law is a small-deformation approximation, and real springs deviate as forces grow. So the data is *approximately* linear, but not exactly — raising the central question of how to extract the best line (or curve) from imperfect measurements.

Fitting Curves to Data

Curve fitting is the process of constructing a curve, or mathematical function, that has the best fit to a series of data points, possibly subject to constraints. In our setup, the fit relates an **independent variable** — the mass, which the experimenter chooses — to an estimated value of a **dependent variable** — the distance, which responds. Fitting comes in two flavors: *interpolation*, where an exact pass through every data point is required, and *smoothing*, where a smooth function is constructed that only approximately fits. A closely related topic, regression analysis, focuses on statistical inference — quantifying how much uncertainty a fitted curve carries when the data contain random errors. Fitted curves serve to visualize data, to infer function values where no data exist, and to summarize relationships among variables; using a fit *beyond* the observed range (extrapolation) is expressly risky, since it may reflect the fitting method as much as the data.

How the “best” fit is defined depends on context. For linear-algebraic analysis, fitting usually means minimizing the *vertical* (y-axis) displacement of points from the curve — ordinary least squares. Graphical and image applications instead pursue *geometric* fitting, minimizing orthogonal distance to the curve (total least squares); results look more aesthetically and geometrically accurate, but such fits are unpopular because they demand nonlinear, iterative computation.

A key structural fact concerns polynomials: a degree- n polynomial exactly fits $n + 1$ constraints. A line (degree 1) passes exactly through any two points with distinct x coordinates; a quadratic fits three points; a cubic fits four. Constraints need not be points — they can be angles or curvatures, often imposed as end conditions to blend polynomial segments smoothly into splines (a technique used, e.g., in highway cloverleaf design, where controlling the rate of change of curvature keeps jerk — and thus forces on the car — reasonable enough to set speed limits). When there are *more* than $n + 1$ constraints, an exact fit is generally impossible, and some method must compare approximations — the least squares method being the standard way to score the deviations. There are good reasons to prefer an approximate low-degree fit over inflating the degree to force an exact match: unnecessarily high-degree polynomials are undesirable, and too few constraints yield infinitely many solutions (a line pinned to a single point), creating an ill-posed selection problem. Hence the rule: choose the lowest degree that works, or lower still if an approximate fit is acceptable. Beyond polynomials, other families suit other phenomena — trigonometric functions for periodic data, Gaussian/Lorentzian/Voigt profiles in spectroscopy, logistic curves for population growth and epidemic spread, inverted logistic sigmoids for crop yield versus soil salinity (slow yield loss at low salinity, accelerating thereafter), parabolas for gravity-driven trajectories, sinusoids for tides.

The Objective Function: Quantifying Goodness of Fit

Deciding *how well* a curve fits requires a measure of goodness, called the **objective function** — a term that will recur throughout the course. Once defined, the goal is to find the curve that **minimizes** it. Concretely, since the underlying relationship here is believed linear, we seek a line such that some function of the sum of the distances from the line to the measured points is as small as possible: draw a line through the cloud of points and choose it to make the total discrepancy minimal. The open question — deferred to the next segment — is exactly *which* function of those distances to use.

That question is the province of **loss functions**. In mathematical optimization and decision theory, a loss (or cost, or error) function maps an event or values of one or more variables onto a real number intuitively representing the “cost” of that event; an optimization problem seeks to minimize it. An objective function is either a loss function or its opposite — a reward, profit, utility, or fitness function — in which case it is

maximized. In statistics, loss functions drive parameter estimation, with the event being some function of the difference between estimated and true values; the concept dates back to Laplace and was reintroduced by Abraham Wald in the mid-20th century. Different fields instantiate it differently: economic cost or regret in economics, penalties for misclassification in classification, benefits-versus-premiums modeling in actuarial science (from Harald Cramér’s work in the 1920s), deviation-from-target penalties in optimal control, and monetary loss in financial risk management.

The most common choice for fitting is the **quadratic loss** — for target t and constant C ,

$$L(\text{error}) = C \cdot (\text{error})^2,$$

the squared error loss underlying least squares. Its popularity is earned: it is mathematically tractable thanks to the properties of variances, and it is *symmetric* — an error above the target incurs the same loss as an equal-magnitude error below it (the constant C is irrelevant to decisions and set to 1). Much of classical statistics — t -tests, regression models, design of experiments — rests on least squares applied through linear regression theory, and linear-quadratic optimal control stays tractable precisely because quadratic loss yields linear first-order conditions. But squaring cuts both ways: quadratic loss assigns disproportionate importance to large errors, so a few outliers can dominate the fit. When data contain many large outliers, robust alternatives such as the Huber, log-cosh, and SMAE losses are preferred; elsewhere a simple 0–1 loss (Hamming distortion in information theory) counts outright mistakes. This sensitivity to outliers is not academic — it bears directly on our spring data, where measurement noise already produced visibly anomalous points, and the choice of discrepancy function will determine how much those points drag the fitted line around.

Measuring Error: Which Distance Is the Right One?

Before any curve can be fit, we must settle what “distance from the curve” even means. For a point sitting off a fitted line there are three candidates: the **horizontal** distance (across to the line at the point’s own height), the **vertical** distance (straight down to the line at the point’s own x), and the **perpendicular** distance (the shortest possible path to the line). Even though the perpendicular path is geometrically shortest, it is the wrong quantity here. The entire purpose of the model is prediction: given an independent value x , produce the dependent value y . The vertical gap is exactly the error of that prediction — the difference between what the model outputs at that x and what was actually observed. The horizontal and perpendicular distances measure no such error, so fitting procedures measure error vertically.

Statistics formalizes this vertical gap as the **residual**. For a model function $f(x, \beta)$ holding m adjustable parameters in the vector β , the residual at an observed data pair (x_i, y_i) is

$$r_i = y_i - f(x_i, \beta),$$

the observed value minus the value predicted by the model. The fit of the model to a data point is measured by this residual, and the fit to the whole data set by how the residuals combine.

The Least-Squares Objective Function

With vertical error chosen, the natural objective is to minimize the sum of squared residuals. For observations indexed $i = 0, \dots, n - 1$:

$$S = \sum_{i=0}^{n-1} (\text{observed}_i - \text{predicted}_i)^2$$

For every data point, take the difference between what was observed and what the model predicted, square it, and add everything up. Adjusting the model’s parameters to make S as small as possible is precisely the **method of least squares**, the standard way to determine a best-fit model in regression analysis.

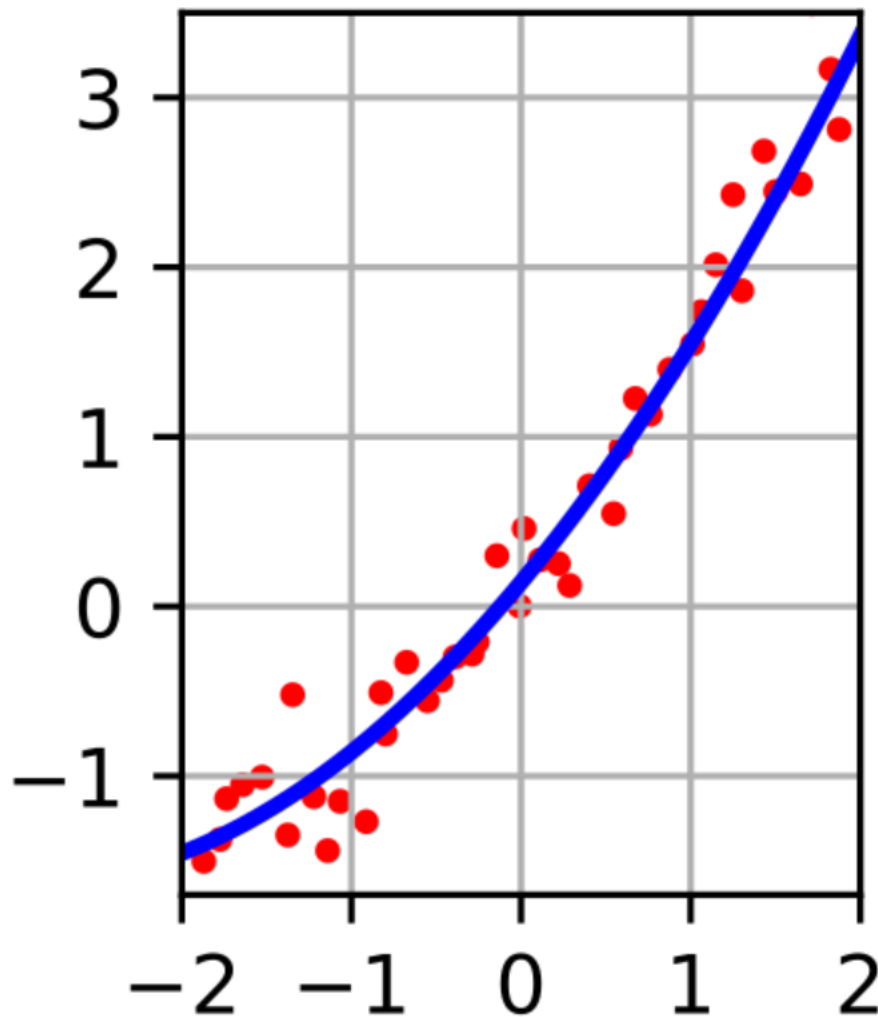
This objective is not arbitrary — it is intimately connected to variance. Variance is the *average* of squared deviations; the sum above is the same computation without dividing by the count, i.e., variance times the number of observations. Whatever parameter values minimize the sum therefore also minimize the variance of the errors, tying least squares to a quantity you already know how to reason about. The degenerate case makes this vivid: if the model is simply a constant, $f(x_i, \beta) = \beta$, the least-squares solution is the arithmetic mean of the input data — the value that minimizes total squared deviation is exactly the sample mean. Least squares generalizes “averaging” to arbitrary fitted curves.

Squared error is also statistically principled, not just convenient. When the observations come from an exponential family with the identity as its natural sufficient statistic and mild conditions hold (as for the normal, exponential, Poisson, and binomial distributions), least-squares estimates coincide with maximum-likelihood estimates. The method’s track record explains why it dominates: Legendre published the first clear exposition in 1805, and within ten years it was standard in astronomy and geodesy across France, Italy, and Prussia. Gauss, who claimed the method since 1795, connected it to probability theory — asking what error density would make the arithmetic mean optimal led him to invent the normal distribution — and used it to predict the position of the asteroid Ceres after it was lost behind the Sun in 1801; his were the only predictions that allowed von Zach to relocate it. In 1822 Gauss established the method’s optimality: when errors have zero mean, are uncorrelated, normally distributed, and share equal variances, the least-squares estimator is the best linear unbiased estimator of the coefficients — the result extended as the **Gauss–Markov theorem**.

Polynomials: The Family of Candidate Models

To apply least squares we need a family of curves to fit. A polynomial in one variable x is zero, or a finite sum of non-zero terms, each of the form cx^p where c is a coefficient (a real number) and p is the degree of the term (a non-negative integer). The degree of the whole polynomial is simply the largest degree among its terms. The familiar examples sit at degrees one and two: a line is $ax + b$, a parabola is $ax^2 + bx + c$.

Fitting these models by least squares is common enough to carry its own name — polynomial least squares — which characterizes how the variance in the prediction of the dependent variable depends on the independent variable and on the deviations from the fitted curve. A degree-two fit produced this way looks like:



Source: Wikipedia, article "[Least squares](#)".

Linear Regression: The Machinery That Solves Least Squares

Minimizing the least-squares objective means finding the curve whose predictions make S minimal, and the tool for this is **linear regression**. Linear regression models the relationship between a scalar response (the dependent variable) and one or more explanatory variables (regressors, or independent variables): with exactly one explanatory variable it is *simple* linear regression, with two or more it is *multiple* linear regression (distinct from multivariate regression, which predicts several correlated dependent variables at once). It was the first type of regression studied rigorously and used extensively, because models that depend linearly on their unknown parameters are easier to fit, and the statistical properties of their estimators are easier to determine.

The model assumes the relationship between y and the regressors is linear, plus a disturbance or error term ε — an unobserved random variable that adds noise to the relationship:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \beta + \varepsilon_i, \quad i = 1, \dots, n.$$

The n equations stack into matrix form $\mathbf{y} = X\beta + \varepsilon$, and fitting amounts to estimating the coefficients β so that the error $\varepsilon = \mathbf{y} - X\beta$.

Automating the fit: `pylab.polyfit`

Deriving least-squares formulas by hand, as done earlier in the lecture, is instructive but unnecessary in practice: Pylab ships a built-in solver. The call `pylab.polyfit(xVals, yVals, n)` takes the observed x values, the observed y values, and a degree n , and returns the coefficients of the degree- n polynomial giving the best least-squares fit to the data. With $n = 1$ it produces the best line $y = ax + b$; with $n = 2$ the best parabola $y = ax^2 + bx + c$. This is *exactly* the optimization problem set up by hand previously — Pylab simply performs the work of finding the coefficients.

The underlying problem is well-defined and solvable in closed form: least squares determines the best-fit model by minimizing the sum of squared residuals, where each residual is the gap between an observation and the model's prediction,

$$r_i = y_i - f(x_i, \beta), \quad S = \sum_{i=1}^n r_i^2.$$

Because the linear least-squares problem admits a closed-form solution, no iterative search is needed here. Polynomial fitting specifically benefits from a structural fact: although a polynomial model is nonlinear in x , the regression function is *linear in the unknown parameters* $\beta_0, \beta_1, \dots$, so polynomial regression is a special case of multiple linear regression — one treats $x, x^2, \dots$ as distinct independent variables. In matrix form the estimates come from a design matrix $\mathbf{X}$; since $\mathbf{X}$ is a Vandermonde matrix, the required invertibility is guaranteed whenever all the x_i are distinct, yielding the unique least-squares solution.

The spring experiment: from masses to forces, and recovering k

The function `fitData(fileName)` wires `polyfit` into the physical experiment:

1. Read the x and y values from the file (`getData`) and convert them to Pylab arrays.
2. Apply the physics: multiply the x values by 9.81, converting hung masses into forces, since force is mass times gravitational acceleration, $F = mg$.
3. Plot the measured points as blue circles labeled “Measured points” and tidy the axes with `labelPlot`.
4. Call `pylab.polyfit(xVals, yVals, 1)` for a linear fit, unpacking the two returned coefficients into `a` and `b`.
5. Compute estimated y values as `a*xVals + b` — the note in the code that this involves a redundant array conversion is correct, because multiplying an array by a scalar and adding a scalar already works elementwise.
6. Print the coefficients and draw the fit as a red line whose legend reports the spring constant as $k = 1/a$, rounded to five decimal places, then place the legend wherever Pylab deems best.

Why $k = 1/a$? Hooke's law states that the force needed to extend or compress a spring by a distance x scales linearly with that distance,

$$F_s = kx \quad \Longleftrightarrow \quad x = \frac{F_s}{k},$$

where k is a positive constant characteristic of the spring's stiffness. Consequently the graph of applied force versus displacement is a straight line whose slope is k — or, read the other way around as the lecture plots it (displacement versus force), the slope is $1/k$. The fitted slope a therefore equals $1/k$, and the run reports $k \approx 21.53686$.

The resulting picture fits the data reasonably well over most of the range, but the measured points visibly bend away from the line and flatten out at the higher forces, around 7–9 Newtons. This is not a surprise physically: Hooke’s law is explicitly a *first-order linear approximation* to the real response of springs, valid only while the deformation is small compared to the spring’s total possible deformation. No material can be stretched beyond a maximum size without permanent deformation, and many materials deviate noticeably well before those elastic limits are reached. The flattening at high load is the spring leaving its Hookean regime — a fact worth keeping in mind for later in the lecture.

pylab.polyval: separating the model from its evaluation

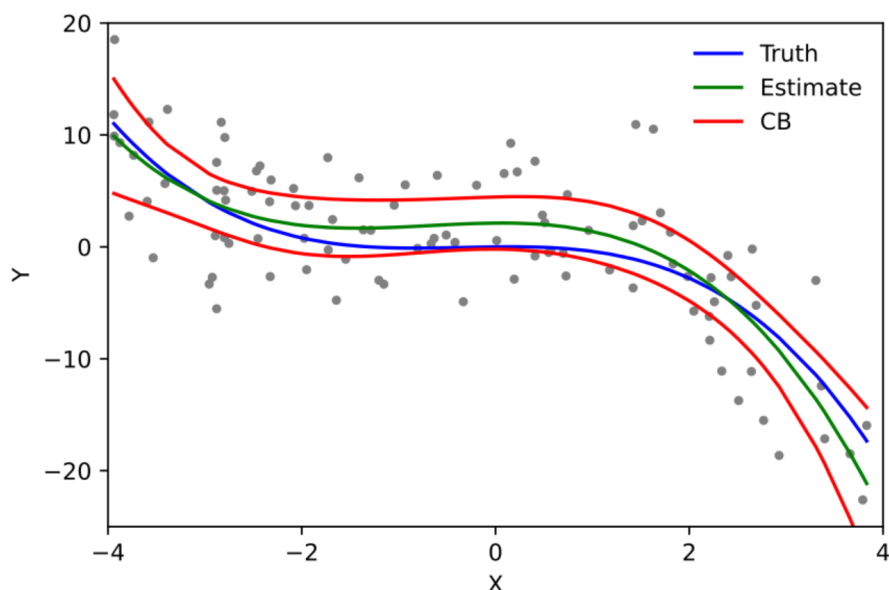
A cleaner variant, `fitData1`, is identical down through `labelPlot` but restructures the fitting step. Instead of unpacking coefficients into named variables and writing out the polynomial manually, it stores whatever `polyfit` returns in a single variable `model`, then calls `pylab.polyval(model, xVals)`. `Polyval` evaluates the polynomial described by those coefficients at each `x`, producing exactly the same `estYVals` as before.

The payoff is generality. If tomorrow you want a parabola instead of a line, you change the degree argument from 1 to 2 and `polyval` still works — no formula rewrite required. The only other edit is in the plot label, which uses `model[0]` (the leading coefficient) rather than the named variable `a` to compute $1/\text{slope}$ for reporting k . The pattern — `polyfit` to find a model, `polyval` to apply it — decouples *choosing* a model class from *evaluating* it.

Choosing the degree by reading the data’s shape

With these tools in hand, the lecture turns to mystery data with a distinctive geometry: the points start high on the left near 300, descend through a dip to roughly -50 near $x = 0$, and climb back to almost 300 on the right. That U-shape is very clearly not a line — a straight-line fit would miss almost every point. But the shape itself signals the remedy: it suggests precisely which degree polynomial to fit, namely a quadratic.

This is the standard reasoning of polynomial regression. When a linear model cannot hold — the canonical example being a chemical yield that improves over one temperature range but decreases over another — a quadratic model captures the reversal, because the effect of a unit increase in x is $\beta_1 + 2\beta_2x$: it depends on where you are along the axis, which is exactly what makes the relationship nonlinear in x even though estimation remains linear in the parameters. In general the expected value of y can be modeled as an n th-degree polynomial, and the degree is chosen to match the qualitative behavior the data exhibits. A fitted polynomial of degree three on scattered data shows how the curve bends to follow structure a line cannot:



Source: Wikipedia, article “[Polynomial regression](#)”.

That is precisely the move the lecture makes next: inspect the mystery data, recognize the U, and reach for degree 2.

A dataset that bends: where the straight line fails

Slide 21 puts the fitting recipe to work on “mystery data”: a scatter of blue points with x running from -10 to 10 and y spanning roughly -70 up to 300 . Following the recipe developed earlier, the professor asked `polyfit` for the best degree-one model and plotted the result in green. What came back is almost perfectly flat, sitting at roughly 100 across the entire range.

Is that a good summary of the data? Not really. The points clearly dip down in the middle and climb back up at both ends — there is a curve living in this data, and a straight line simply cannot bend. The line is not catastrophically wrong, but it is obviously missing the structure. The lesson is that the fitting procedure will faithfully return the *best line* you asked for, even when the underlying phenomenon is curved; choosing the degree of the polynomial is itself a modeling decision, and a poor one caps how well any fit can describe the data.

Fitting a quadratic — and why it is still linear regression

The fix is a higher-degree model, and the code change is minimal. On slide 22 the only difference from before is circled in red:

```
model2 = pylab.polyfit(xVals, yVals, 2)
```

That third argument is the **degree** of the polynomial, so instead of the best line we are now asking for the best parabola. To draw it, `pylab.plot(pylab.polyval(model2, xVals), ...)` evaluates the polynomial that `polyfit` handed back at each x value (`polyval` simply plugs the x ’s into the fitted coefficients), rendered as a red dashed line labeled ‘Quadratic Model’.

Slide 23 shows all three ingredients together: the blue data points, the green solid line for the linear model, and the red dashed curve for the quadratic. The quadratic is visibly the better description: it comes in high on the left around 260 , sweeps down through the middle of the point cloud, bottoms out near zero around

$x = 0$, and climbs again on the right, tracking the points as they rise. The green line, by contrast, just plows straight through the middle, ignoring the bend entirely.

Here is the subtlety the professor flags as an exam trap: **this is still an example of linear regression**, even though we are not fitting a line. The reason is that the model is *linear in the unknown coefficients*. As the linear regression literature puts it, linear regression models the relationship between a scalar response and explanatory variables using **linear predictor functions whose unknown model parameters are estimated from the data**:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^\top \beta + \varepsilon_i,$$

where ε is a disturbance term adding noise to the relationship, and fitting means estimating the β 's so that the error $\varepsilon = \mathbf{y} - \mathbf{X}\beta$ is small — commonly by minimizing the sum of squared errors $\|\varepsilon\|_2^2$. Nothing in this framework requires the regressors x_{ij} to be the raw input; they can be any functions of it. The classic illustration is a ball tossed in the air: physics says its height is quadratic in time, h involving t and t^2 , and while that model is *non-linear in the time variable*, it is *linear in the parameters* β_1, β_2 — take regressors $x_i = (t_i, t_i^2)$ and it drops into the standard form above. Our quadratic fit is exactly this construction with regressors built from x and x^2 : two explanatory variables, which by the standard taxonomy makes it a *multiple* linear regression (one explanatory variable would be *simple*; “multivariate” refers instead to predicting several correlated responses). “Linear” refers to how the coefficients enter the equation, not to the shape of the curve that gets drawn. This linearity is precisely why such models were the first type of regression studied rigorously and used extensively: models linear in their parameters are easier to fit than ones related non-linearly to their parameters, and the statistical properties of the resulting estimators are easier to determine. (Relatedly, the terms “least squares” and “linear model” are closely linked but not synonymous — least-squares machinery can even fit models that are not linear.)

Two distinct questions about goodness of fit

Visual agreement is not evaluation — “appears to be a better fit” leaves open how good the fits actually are. Before addressing that, slide 24 quietly changes the axes: x now runs from -60 to 60 and y from -4000 up to 8000 . Same data, same two models — the green line still sits nearly flat (around 2400 on this scale) and the red parabola still dips through the middle — but suddenly they are displayed on a much bigger canvas. The professor explicitly asks you to keep this in mind, as it will matter shortly.

The question itself splits into two, and they require different tools:

- **Relative:** which of these fits is better than the other?
- **Absolute:** how good is each fit on its own merits?

The relative question comes first.

Relative comparison via mean squared error

A fit, remember, is a function from the independent variable to the dependent variable: hand it a value of x and it returns an estimate of y — estimation is its entire job. So asking which fit is better really asks *which fit provides better estimates*. And there is a natural yardstick, because of how the fits were built in the first place: each one was found by **minimizing mean square error** — the average of the squared differences between the observed y values and the model's predictions. Evaluating that same error for each model and comparing gives the relative ranking.

The statistics behind this quantity are worth knowing precisely. The **mean squared error (MSE)**, also called the mean squared deviation, measures the average of the squares of the errors — the average squared difference between estimated values and true values. It is a *risk function*, corresponding to the expected value of squared-error loss; in machine learning's empirical-risk-minimization framing, the MSE computed on an observed data set is the *empirical risk*, an estimate of the true risk averaged over the actual population distribution. Several structural facts follow:

- Because it derives from the square of Euclidean distance, MSE is **always positive** and decreases as error approaches zero; it is almost never exactly zero, either because of randomness or because the estimator fails to account for information that could produce a more accurate estimate.
- Its units are the **square** of the quantity being estimated. Taking its square root yields the **root-mean-square error (RMSE)**, which is back in the original units — for an unbiased estimator, the RMSE is the standard error.
- MSE is the second moment of the error, so it folds together two things: the **variance** of the estimator (how widely estimates spread from sample to sample) and its **bias** (how far the average estimate sits from the truth):

$$\text{MSE}(\hat{\theta}) = \text{Var}_{\theta}(\hat{\theta}) + \text{Bias}(\hat{\theta}, \theta)^2,$$

so for an unbiased estimator the MSE *is* the variance.

For judging a fitted model — which is a *predictor*, a function mapping inputs to predicted values, rather than an *estimator* of a single population parameter — the relevant version is the **within-sample MSE**. Given n observations Y and predictions $\hat{Y}$ (for instance from a least-squares fit),

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 = \frac{1}{n} \mathbf{e}^T \mathbf{e},$$

where $\mathbf{e}$ is the vector of residuals $Y_i - \hat{Y}_i$. This is easily computable for a particular sample, though it is inherently sample-dependent. The same formula applied to q points **not** used in fitting the model — either held back or newly obtained — gives the **test MSE**,

$$\text{MSE} = \frac{1}{q} \sum_{i=n+1}^{n+q} (Y_i - \hat{Y}_i)^2,$$

a process known as cross-validation; it separates how well a model does on the data it was tuned to from how well it generalizes.

Why is comparing MSEs the right move for the relative question? Because both candidate models were produced by minimizing this exact objective on the same data, their achieved MSEs rank them by the very criterion they were built to optimize: the model with lower average squared estimation error is, under this loss, the one providing better estimates. One caveat from the regression literature tempers enthusiasm for squared error: because squaring assigns disproportionately high importance to large errors, an MSE cost on data with many big outliers can produce a model that fits the outliers more than the true signal — which is why robust cost functions (such as least absolute deviations) or penalized variants like ridge (L_2 penalty) and lasso (L_1 penalty) exist as alternatives.

Quantifying Fit Error: Mean Squared Error

Once two candidate models exist — a linear fit (`model1`) and a quadratic fit (`model2`) — we need a number that says which is better. The function `aveMeanSquareError(data, predicted)` does exactly this: it loops over every data point, accumulates the square of the difference between the observed value and the predicted value, $(data[i] - predicted[i])^2$, and at the end divides the accumulated total by the number of data points. Predictions themselves come from applying a fitted model to the x-values with `pylab.polyval`. Running this on both models gives:

- Linear model: average mean square error ≈ 9372.73078965
- Quadratic model: average mean square error ≈ 1524.02044718

The quadratic model's error is roughly six times smaller, confirming quantitatively what was visible qualitatively: the quadratic curve hugs the data much more closely than the straight line.

This quantity is the **mean squared error (MSE)**, defined for n predictions $\hat{Y}_i$ against observed values Y_i as

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2,$$

i.e., simply the mean of the squared errors. Several properties follow from its construction, and they matter for interpreting the numbers above:

- **Always positive.** MSE is derived from the square of Euclidean distance, so it is always a positive value that decreases as the error approaches zero. It is almost never exactly zero because of randomness in the data or because the estimator fails to account for information that would produce a more accurate estimate.
- **A risk function.** MSE corresponds to the expected value of the squared error loss — the quantity minimized, at least empirically, when we fit by least squares.
- **Units get squared.** Like the variance, MSE carries the *square* of the measurement units. Taking its square root yields the **root-mean-square error (RMSE)**, which restores the original units — analogous to how the standard deviation relates to the variance. For an unbiased estimator, the RMSE is the standard error.
- **It blends two failure modes.** As the second moment of the error, MSE incorporates both the *variance* of the estimator (how widely estimates spread across samples) and its *bias* (how far the average estimate sits from the truth). Indeed, it decomposes exactly as

$$\text{MSE}(\hat{\theta}) = \text{Var}_{\theta}(\hat{\theta}) + \text{Bias}(\hat{\theta}, \theta)^2,$$

so for an unbiased estimator the MSE *is* the variance. Squaring the errors is precisely what lets one number capture both random scatter and systematic offset.

One further distinction matters for later lectures: the within-sample MSE above is computed on the very points used to fit the model. The MSE can also be computed on q points held out of the fit (or newly collected) — the **test MSE**

$$\text{MSE} = \frac{1}{q} \sum_{i=n+1}^{n+q} (Y_i - \hat{Y}_i)^2,$$

which is the basis of cross-validation. Within-sample error flatters flexible models; held-out error is the honest check.

From Relative Comparison to Absolute Assessment: The Coefficient of Determination

MSE answered “which model is better?” cleanly. But slide 27 asks a harder question: is 1524 *good* in an absolute sense? There is no way to tell, for two structural reasons:

1. **No upper bound.** Mean squared error can be arbitrarily large, so there is no “perfect score” to compare against.
2. **Not scale independent.** Measuring the same quantities in different units changes the error even though the quality of the fit is unchanged.

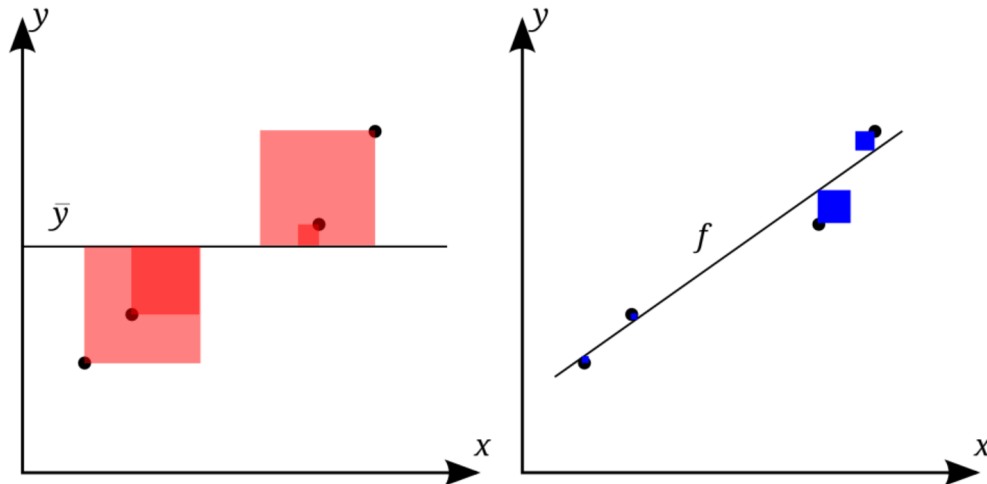
The fix is the **coefficient of determination**, R^2 . Formally, given measured values $y_1, \dots, y_n$, predicted (fitted) values $p_1, \dots, p_n$, and the mean of the measurements $\mu = \frac{1}{n} \sum_{i=1}^n y_i$, define

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - p_i)^2}{\sum_i (y_i - \mu)^2}.$$

The logic of the ratio is the whole point:

- The **numerator** (SS_{res} , built from residuals $e_i = y_i - p_i$) measures the *error in the estimates* — variation left unexplained by the model.
- The **denominator** (SS_{tot}) measures the *variability inherent in the measured data itself* — the spread around the mean that any model would have to explain.

So R^2 compares the estimation errors against the variability of the original values, capturing **the proportion of the variability in the data set that is accounted for by the statistical model provided by the fit**. It is a dimensionless ratio, which dissolves both objections to MSE: it cannot grow without bound relative to its denominator, and it is invariant to rescaling of the units.



Source: Wikipedia, article “[Coefficient of determination](#)”.

The picture makes the ratio concrete: red squares represent the squared residuals with respect to the simple average (the denominator, SS_{tot}), while blue squares represent the squared residuals with respect to the regression line (the numerator, SS_{res}). The better the regression fits compared to just predicting the average, the smaller the blue area relative to the red, and the closer R^2 is to 1.

Two anchor cases fall out immediately. In the best case the modeled values exactly match the observations, so $SS_{\text{res}} = 0$ and $R^2 = 1$. At the other extreme, a **baseline model that always predicts the mean $\bar{y}$** has $R^2 = 0$: it explains none of the variability. In general $R^2 = 1 - \text{FVU}$, where the fraction of variance unexplained (FVU) compares the variance of the model’s errors to the total variance of the data — a larger R^2 always implies a more successful regression model.

Implementing R^2 : The Variance Shortcut

Slide 28 turns the formula into a three-line function:

```
def rSquared(observed, predicted):
    meanError = sum((predicted - observed)**2)/len(observed)
    return 1 - meanError/numpy.var(observed)
```

The brevity hides a deliberate algebraic trick. The textbook ratio has sums in both numerator and denominator:

- Numerator: $SS_{\text{res}} = \sum_i (y_i - p_i)^2$. Dividing by the number of samples n gives the **average sum-squared-error** — that is `meanError`.
- Denominator: $SS_{\text{tot}} = \sum_i (y_i - \mu)^2$. Dividing by the same n gives exactly the **variance** of the observed data — that is `numpy.var(observed)`.

Since the same factor n appears in numerator and denominator, it cancels:

$$R^2 = 1 - \frac{SS_{\text{res}}/n}{SS_{\text{tot}}/n} = 1 - \frac{\text{mean squared error}}{\text{variance of the data}},$$

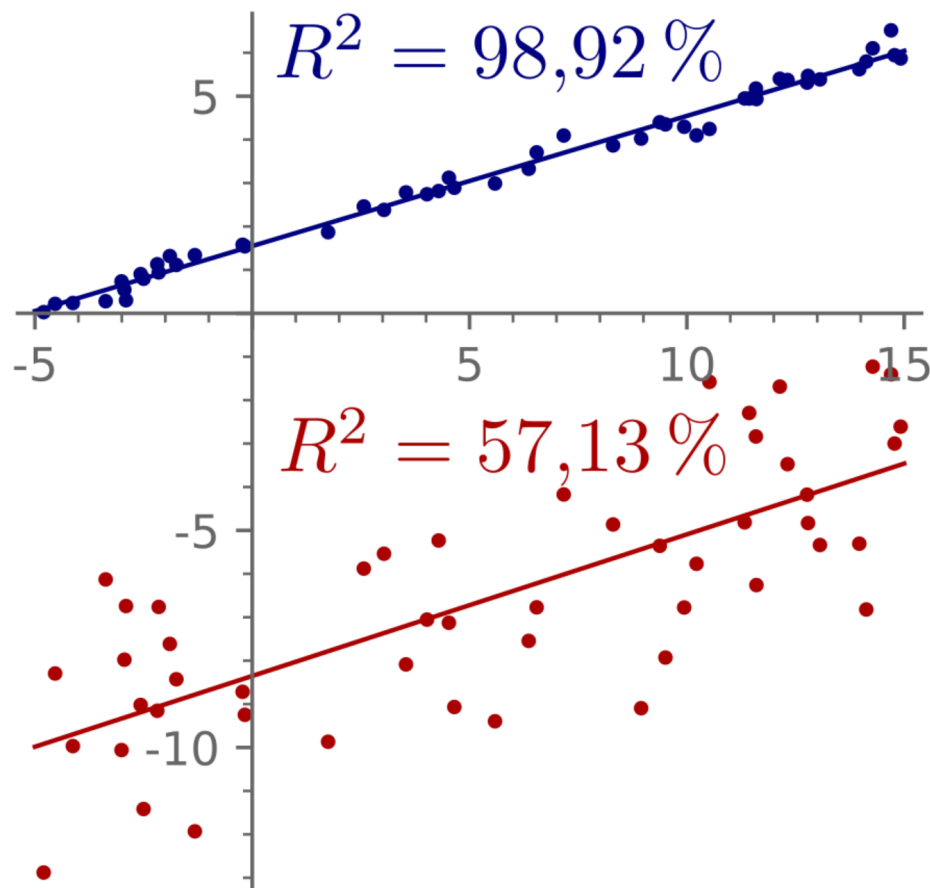
so the ratio of the mean error to the variance is *identical* to the ratio of sums in the formal definition. No separate loop over the mean is needed.

What R^2 Means — and When to Distrust It

Interpretation is where R^2 earns its keep. By comparing estimation errors to the data's own variability, R^2 reads directly as a share of explained variation:

- $R^2 = 1$: the model explains **all** of the variability in the data.
- $R^2 = 0$: there is **no relationship** between the values predicted by the model and the actual data — the model tells you nothing beyond the mean.
- $R^2 = 0.5$: the model explains **half** the variability.

This gives the scale intuition that raw MSE lacked: 1524 meant nothing to us, but an R^2 of, say, 0.9 immediately says something meaningful. Concretely, an R^2 of 0.49 means 49% of the variability of the dependent variable has been accounted for and the remaining 51% is still unexplained.



Source: Wikipedia, article "[Coefficient of determination](#)".

Because it is a percentage-like, bounded quantity, R^2 is typically **more intuitively informative than MAE, MAPE, MSE, and RMSE**, whose ranges are arbitrary, and it proved more robust than SMAPE for poor fits on certain test datasets.

There are also useful equivalences to know. In **simple linear regression with an intercept**, r^2 is simply the square of the sample correlation coefficient between the observed outcomes and the predictor values; with additional regressors, R^2 becomes the square of the coefficient of multiple correlation. More generally, in linear least squares with fitted intercept and slope, R^2 equals $\rho^2(y, f)$ — the squared Pearson correlation between observed and modeled values — and with a single explanatory variable this reduces to $\rho^2(y, x)$, the squared correlation between y and x . Underlying this is a sum-of-squares partition: when the fitted values come from linear regression, the total sum of squares splits as $SS_{\text{tot}} = SS_{\text{reg}} + SS_{\text{res}}$ (explained plus residual), and R^2 can equivalently be read as the ratio of explained variance to total variance.

Two caveats temper the clean $[0, 1]$ story:

- **The bound is conditional.** R^2 lies between 0 and 1 when the fit is generated by a linear regression (with intercept) and tested on the *training* data. Negative values can arise when the predictions were not derived from a model-fitting procedure using those data, when linear regression omits the intercept, or when a non-linear function is used to fit the data. In all such cases a negative R^2 signals that, by this criterion, **the mean of the data provides a better fit than the fitted function values do.**
- **Don't confuse correlation with agreement.** When evaluating simulated versus measured values, it is inappropriate to base goodness-of-fit on the R^2 of the regression $Y_{\text{obs}} = m \cdot Y_{\text{pred}} + b$: that R^2 quantifies the degree of *any* linear correlation between the two series. Proper goodness-of-fit demands the specific 1:1 relationship $Y_{\text{obs}} = 1 \cdot Y_{\text{pred}} + 0$.

A Reusable Harness for Comparing Fits of Many Degrees

Slide 30 packages everything into two functions so that goodness-of-fit can be tested systematically rather than model-by-model:

- `genFits(xVals, yVals, degrees)` loops over a list of polynomial degrees, calls `pylab.polyfit(xVals, yVals, d)` for each degree d , appends each resulting model to a list, and returns the list of models. One call produces a whole family of candidate fits.
- `testFits(models, degrees, xVals, yVals, title)` first plots the raw data as circles labeled 'Data'. Then, for each model, it uses `pylab.polyval` to compute the estimated y-values, scores the fit with the `rSquared` function, and plots the fit with a label stating both the degree of the polynomial and its R^2 rounded to five decimal places. It finishes by placing the legend in the best location and titling the figure.

Together they let us generate fits of many degrees and, in a single picture, compare both how each curve looks against the data and how each scores by R^2 — the machinery needed to ask, in a principled way, whether pushing to higher-degree polynomials keeps buying real explanatory power.

Measuring Explanatory Power: The Coefficient of Determination

Once you can fit curves to data, the natural question is: how do you know a fit is *good*? One powerful way to frame the question is to ask how much of the variance in the data the model actually explains. Statistics answers this with the **coefficient of determination**, denoted R^2 or r^2 : the proportion of the variation in the dependent variable that is predictable from the independent variable(s). It measures how well observed outcomes are replicated by the model, and it is used in statistical models whose purpose is prediction of future outcomes or hypothesis testing.

Concretely, suppose a dataset has n observed values $y_1, \dots, y_n$, each associated with a fitted (modeled) value $f_1, \dots, f_n$. Define the residuals $e_i = y_i - f_i$ and the mean of the observed data,

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

The variability of the dataset is captured by sums of squares, and the most general definition of the coefficient is

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

In the best case the modeled values exactly match the observed ones, so $SS_{\text{res}} = 0$ and $R^2 = 1$; a baseline model that always predicts $\bar{y}$ achieves $R^2 = 0$. Equivalently, $R^2 = 1 - \text{FVU}$, where the second term compares the unexplained variance (variance of the model's errors) against the total variance of the data. As an interpretive anchor: $R^2 = 0.49$ means 49% of the variability of the dependent variable has been accounted for, and 51% remains unaccounted for; a larger value implies a more successful regression model.

Several structural facts make R^2 convenient. In simple linear regression with an intercept, r^2 is simply the square of the sample correlation coefficient between observed outcomes and predictor values; with additional regressors it becomes the square of the coefficient of multiple correlation. In linear least squares regression with fitted intercept and slope, R^2 equals $\rho^2(y, f)$, the squared Pearson correlation between observed and modeled values (and, with a single explanator, $\rho^2(y, x)$). When the usual sum-of-squares partition holds — as in ordinary linear regression — R^2 can also be read as the ratio of explained variance (SS_{reg}/n) to total variance (SS_{tot}/n). Note that negative values *can* arise: when predictions were not derived from a model-fitting procedure using those data, when regression omits the intercept, or when a nonlinear function is fit — in such cases the mean of the data provides a better fit than the fitted function under this criterion.

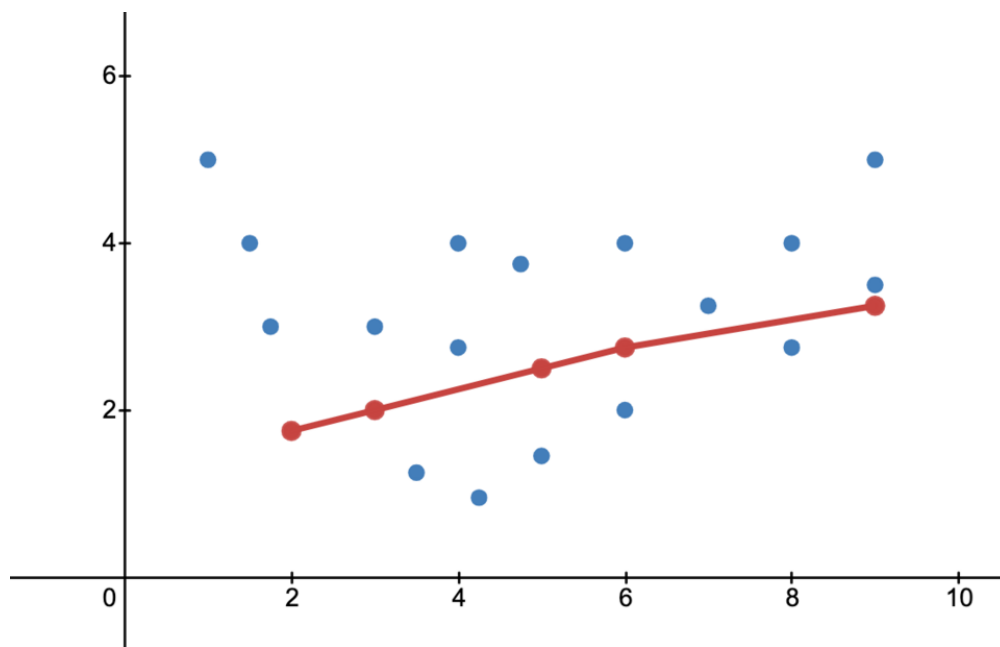
Why report R^2 instead of MAE, MAPE, MSE, or RMSE? Because it can be expressed as a percentage, whereas those measures have arbitrary ranges; it also proved more robust than SMAPE for poor fits on certain test datasets. One caution from the literature: when evaluating goodness-of-fit of simulated (Y_{pred}) versus measured (Y_{obs}) values, it is *not* appropriate to use the R^2 of the linear regression $Y_{\text{obs}} = m \cdot Y_{\text{pred}} + b$, because that quantifies *any* linear correlation, whereas goodness-of-fit demands specifically the 1:1 line $Y_{\text{obs}} = 1 \cdot Y_{\text{pred}} + 0$.

Applied to the lecture's mystery data (blue dots spanning x from -10 to 10), a degree-1 polynomial — a straight line — comes out essentially flat at around 100 across the whole range, with $R^2 = 0.00049$: essentially zero. Under the constraint of a straight line, knowing the x -value tells you almost nothing about the y -value.

Underfitting: When the Model Misses the Structure

The flat-line result above is a textbook case of **underfitting**: a mathematical model that cannot adequately capture the underlying structure of the data. An under-fitted model is missing some parameters or terms that would appear in a correctly specified model. The canonical example is exactly what happened here — fitting a linear model to nonlinear data — and such a model tends to have poor predictive performance.

Contrast the degree-2 fit: a parabola captures the U-shape of the data beautifully — high at the edges, dipping down toward zero in the middle — and its R^2 leaps to 0.83748. That single jump, from roughly zero to roughly 84% of the variance explained, is strong evidence that something quadratic is going on in the mystery data. The gap between the two fits is the signature of structure that a too-simple model class simply cannot express.



Source: Wikipedia, article “[Overfitting](#)”.

Climbing the Degree Ladder: Diminishing Returns, Then Trouble

If a parabola does well, why not get a tighter fit by cranking up the polynomial degree? The lecture traces exactly what happens:

Degree	R^2	Character of the fit
1	0.00049	Flat line; explains almost nothing
2	0.83748	Smooth parabola capturing the U-shape
4	0.84895	Barely better than the parabola
8	0.86556	Modest gain; still smooth and parabola-like
16	0.96553	Wiggles wildly, chasing every data point

The pattern is instructive. Going from degree 1 to degree 2 buys enormous explanatory power — the model class finally matches the data’s structure. Beyond that, returns diminish sharply: degrees 4 and 8 add only a few hundredths of R^2 , and their curves remain smooth and parabola-like. But degree 16 pushes R^2 above 96% — at a cost. The magenta curve is no longer a smooth parabola; it swings up and down to chase each individual data point. It is no longer capturing the underlying trend; it is fitting the noise in the data.

This behavior has deep roots in how regression behaves when parameters multiply. In regression analysis, overfitting occurs frequently, and the extreme case is stark: with p variables in a linear regression and p data points, the fitted line can go *exactly* through every point. More generally, if the number of parameters equals or exceeds the number of observations, a model can perfectly predict the training data simply by memorizing it in its entirety — and such a model will typically fail severely when making predictions. Practical guardrails exist for this reason: for logistic regression and Cox proportional hazards models, rules of thumb cap observations per independent variable (guidelines of 5–9, 10, and 10–15; the guideline of 10 observations per variable is known as the “one in ten rule”). For polynomials specifically, the parameters *are* the degree — so every increment of degree is another unit of flexibility available to absorb noise.

Overfitting: When a Fit Memorizes Instead of Learns

Overfitting is the production of an analysis that corresponds too closely or exactly to a particular set of data and may therefore fail to fit additional data or predict future observations reliably. An overfitted model contains more parameters than can be justified by the data. Its essence is to *unknowingly extract some of the residual variation — i.e., noise — as if that variation represented the underlying model structure*. The degree-16 curve is precisely this failure mode made visible.

Why does this happen even to careful analysts? The possibility of overfitting exists whenever the criterion used to *select* a model is not the same as the criterion used to *judge* its suitability. A model might be selected by maximizing its performance on training data, yet its suitability is determined by how well it performs on unseen data. Overfitting occurs when a model begins to “memorize” the training data rather than “learn” to generalize from a trend. In this sense overfitting violates Occam’s razor — including more adjustable parameters than are ultimately optimal, or using a more complicated approach than is ultimately optimal. The contrast is easy to see in miniature: if y can be adequately predicted by a linear function of two independent variables, that function requires only three parameters (an intercept and two slopes); anything richer spends capacity without adding real explanatory power.

Two further consequences deserve attention. First, **shrinkage**: even when the fitted model does *not* have an excessive number of parameters, the fitted relationship should be expected to perform somewhat less well on a new dataset than on the one used for fitting — in particular, the coefficient of determination will shrink relative to the original data. High in-sample R^2 therefore systematically overstates out-of-sample performance. Second, **false discoveries**: with a large set of explanatory variables that actually have no relation to the dependent variable, some will in general be falsely found statistically significant, and a researcher who retains them overfits the model — this is known as Freedman’s paradox.

The modeling literature frames the remedy as balance. Burnham & Anderson argue that to avoid overfitting one should adhere to the **Principle of Parsimony**, noting that overfitted models are often free of bias in their parameter estimators but have needlessly large sampling variances — precision suffers relative to a more parsimonious model — and that false treatment effects tend to be identified and false variables included. Their conclusion: a best approximating model is achieved by properly balancing the errors of underfitting and overfitting. Overfitting is also most likely to be a serious concern when little theory is available to guide the analysis, because then there tend to be a large number of candidate models to select from — “Given a data set, you can fit thousands of models at the push of a button, but how do you choose the best? ... Is the monkey who typed Hamlet actually a good writer?” Formally, the mean squared error of a random regression function decomposes into random noise, approximation bias, and variance in the estimate of the regression function; managing the **bias–variance tradeoff** is the standard lens for overcoming overfit models.

Trading Off Explanation Against Generalization — and What to Do About It

The takeaway of the whole degree-ladder exercise: **higher R^2 is not automatically better**. There is a genuine trade-off between how well a model explains the data we have and how well it will generalize to data we don’t yet have. Keep that in mind whenever you’re tempted to simply keep increasing a model’s complexity.

Fortunately, the literature offers concrete countermeasures. To lessen the chance or amount of overfitting, one can use model comparison, cross-validation, regularization, early stopping, pruning, Bayesian priors, or dropout. These rest on two basic strategies: (1) explicitly penalize overly complex models, or (2) test the model’s ability to generalize by evaluating its performance on a set of data *not* used for training, which is assumed to approximate the typical unseen data a model will encounter.

Strategy (2) has a characteristic visual signature in supervised learning. Training error falls steadily as training proceeds, but if validation error begins to *increase* while training error continues to decrease, overfitting has likely occurred — the model is optimizing for the training set at the expense of generalization. The best predictive model sits at the global minimum of the validation error, not at the point of lowest training error.

Strategy (1) — penalizing complexity, as in regularization — produces a different kind of fit: one that deliberately does *not* trace the training data perfectly. Such a model may look worse on the points it was fit

to, yet achieve lower error on new, unseen data, because it has learned the trend rather than the noise.